

1. What is a REST API?

Answer:

REST (Representational State Transfer) is an architectural style used to build web services.

A REST API allows communication between client and server using HTTP.

□ Example:

- Client (browser/app) → sends request
 - Server → sends response (usually JSON)
-

2. What are HTTP methods in REST?

Answer:

Method	Purpose
GET	Retrieve data
POST	Create new data
PUT	Update entire data
PATCH	Update partial data
DELETE	Delete data

□ Example:

GET /users → get all users

POST /users → create user

3. What is Stateless in REST?

Answer:

REST APIs are **stateless**, meaning:

- Each request is independent
- Server does NOT store client data between requests

So every request must contain all required information (like token, parameters).

4. What is the difference between PUT and PATCH?

Answer:

PUT	PATCH
Updates entire resource	Updates partial resource
Sends full data	Sends only changed fields

□ Example:

PUT /user/1 → update all fields

PATCH /user/1 → update only name

5. What is HTTP status code?

Answer:

Status codes indicate result of request.

Important ones:

Code	Meaning
------	---------

200	Success
-----	---------

201	Created
-----	---------

400	Bad request
-----	-------------

401	Unauthorized
-----	--------------

404	Not found
-----	-----------

500	Server error
-----	--------------

6. What is JSON?

Answer:

JSON (JavaScript Object Notation) is a lightweight format used for data exchange.

☐ Example:

```
{  
  "name": "Shaba",  
  "age": 22  
}
```

7. What is an Endpoint?

Answer:

Endpoint is a URL where API can be accessed.

☐ Example:

<https://api.example.com/users>

8. What is CRUD?

Answer:

CRUD operations:

- Create → POST
 - Read → GET
 - Update → PUT/PATCH
 - Delete → DELETE
-

9. What is Idempotent?

☐ **VERY IMPORTANT (frequently asked)**

Answer:

Idempotent means:

☐ Performing the same request multiple times gives same result

Method Idempotent?

GET Yes

PUT Yes

DELETE Yes

Method Idempotent?

POST No

10. What is Authentication vs Authorization?

Answer:

Authentication	Authorization
-----------------------	----------------------

Who you are	What you can access
-------------	---------------------

Login	Permissions
-------	-------------

☐ Example:

- Login with email → Authentication
 - Access admin page → Authorization
-

11. What is Token-based Authentication?

Answer:

After login:

☐ Server sends a token (JWT)

Client sends token in headers:

Authorization: Bearer <token>

Used for secure APIs.

12. What is REST API best practice?

Answer:

- Use proper HTTP methods
- Use meaningful URLs
- Use status codes correctly
- Keep API stateless
- Use authentication

13. What is Rate Limiting?

Answer:

Restricts number of API requests.

☐ Prevents:

- Server overload
- DDoS attacks

14. What is CORS?

☐ **Very important for frontend interviews**

Answer:

CORS (Cross-Origin Resource Sharing):

☐ Allows frontend (different domain) to access API

Example:

- Frontend: localhost:3000
- Backend: api.com

Without CORS → blocked

15. What is API Versioning?

Answer:

Used to manage API updates.

☐ Example:

/api/v1/users
/api/v2/users

16. What is Query Parameter vs Path Parameter?

Answer:

Path Param Query Param

Required Optional

/users/1 /users?age=20

17. What is Pagination?

Answer:

Used to limit large data.

□ Example:

GET /users?page=1&limit=10

18. What is 3-tier architecture in REST?

Answer:

1. Client (Frontend)
 2. Server (Backend/API)
 3. Database
-

19. What is Content-Type?

Answer:

Specifies data format.

□ Example:

Content-Type: application/json

20. Difference between REST and SOAP?

Answer:

REST	SOAP
-------------	-------------

Lightweight	Heavy
-------------	-------

REST SOAP

Uses JSON Uses XML

Faster Slower

☐ **VERY IMPORTANT CODING QUESTION**

☐ Example (Node.js Express API):

```
const express = require('express');
const app = express();

app.use(express.json());

app.get('/users', (req, res) => {
  res.json([ { name: "Shaba" } ]);
});

app.post('/users', (req, res) => {
  const user = req.body;
  res.status(201).json(user);
});

app.listen(3000, () => console.log("Server running"));
```

☐ **How HTTP works (Simple)**

1. Client sends a request
2. Server processes it
3. Server sends a response

☐ Example:

- You open a website
 - Browser sends HTTP request
 - Server returns HTML/JSON
-

☐ **Example of HTTP Request**

GET /users HTTP/1.1
Host: example.com

❑ Example of HTTP Response

HTTP/1.1 200 OK

Content-Type: application/json

```
{  
  "name": "Shaba"  
}
```

❑ Key Features of HTTP

- ✓ Stateless → each request is independent
 - ✓ Uses request-response model
 - ✓ Works on port 80 (default)
-

❑ HTTP vs HTTPS (IMPORTANT)

HTTP	HTTPS
Not secure	Secure
No encryption	Uses SSL/TLS
Port 80	Port 443

- ❑ In real projects → always use **HTTPS**
-

❑ Short Interview Answer (BEST)

If interviewer asks, say this:

❑

“HTTP stands for HyperText Transfer Protocol. It is used for communication between client and server using a request-response model. It is stateless, meaning each request is independent.”

Difference Between HTTP and HTTPS

❑ Basic Definition

- **HTTP (HyperText Transfer Protocol)** → Not secure
 - **HTTPS (HyperText Transfer Protocol Secure)** → Secure version of HTTP
-

❑ Key Differences (IMPORTANT TABLE)

Feature	HTTP	HTTPS
Security	❑ Not secure	❑ Secure
Encryption	❑ No encryption	❑ Uses SSL/TLS encryption
Data Safety	❑ Data can be intercepted	❑ Data is protected
Port	80	443
URL	http://	https://
Use Case	Basic websites	Banking, login, payments

❑ What actually happens?

HTTP:

- ❑ Data is sent as **plain text**
- ❑ Hackers can read it (Man-in-the-middle attack)

HTTPS:

- ❑ Data is **encrypted**
 - ❑ Even if intercepted → cannot read
-

❑ Example

HTTP:

username=shaba&password=1234

- ❑ Anyone can see this ❑

HTTPS:

- ❑ Data is encrypted → unreadable ❑
-

❑ What is SSL/TLS?

- Security protocol used in HTTPS
- Encrypts data between client & server

□ **Best Interview Answer (SHORT)**

□

“HTTP is not secure and sends data in plain text, while HTTPS is secure and uses SSL/TLS encryption to protect data during communication between client and server.”